Decode Lab

NexaTech

Project 2 Backend Documentation

Node.js + Express.js + MongoDB
Contact Management System

1. Project Overview

Project 2 is the backend component of the NexaTech full-stack application. It provides an API using Node.js and Express.js and stores contact messages in MongoDB through Mongoose. The backend supports creating, viewing, updating and deleting contact records.

2. Project Objectives

- Create a server-side application using Node.js and Express.js.
- Connect the application to MongoDB using Mongoose.
- Receive contact form data from the React frontend.
- Store contact records permanently in the database.
- Return saved contacts to the admin panel.
- Update and delete existing contact records.
- Provide success and error responses for API operations.
- Enable communication between the frontend and backend through HTTP requests.

3. Technologies Used

Technology	Purpose	Project Use
Node.js	JavaScript runtime	Backend execution
Express.js	HTTP server and API routing	server.js
MongoDB	NoSQL database	nexatech database
Mongoose	MongoDB connection/schema/model layer	server.js
CORS	Allows frontend API requests	server.js
dotenv	Environment configuration	.env
JavaScript	Backend programming language	server.js

4. Backend Project Structure

The backend is maintained separately from the React frontend. server.js contains the server, database connection, schema and API routes. .env is used for environment settings and .gitignore prevents sensitive or unnecessary files from being uploaded.

5. Server Configuration

```
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");

const app = express();

app.use(cors());
app.use(express.json());

const PORT = 5000;
```

Express creates the application server. CORS permits communication with the frontend, while express.json() allows JSON request bodies. The backend runs on port 5000.

6. MongoDB Connection

```
mongoose.connect("mongodb://127.0.0.1:27017/nexatech")
.then(() => {
   console.log("MongoDB Connected Successfully");
})
.catch((err) => {
   console.log("MongoDB Connection Error:", err);
});
```

The application connects to the local MongoDB server and uses the database named nexatech. Successful and failed connection states are reported in the terminal.

7. Contact Schema and Model

```
const contactSchema = new mongoose.Schema({
   name: String,
   email: String,
   message: String
});

const Contact = mongoose.model("Contact", contactSchema);
```

The Contact schema defines the fields stored for each message: name, email and message. MongoDB/Mongoose also provides the document _id used for individual update and delete operations.

8. API Endpoints

Method	Endpoint	Purpose	Response
GET	/	Server/home test route	Home Page
GET	/about	About route	About Page
GET	/services	Services route	Services Page
POST	/contact	Create contact	Contact Saved Successfully
GET	/contacts	Fetch all contacts	JSON array
PUT	/contact/:id	Update contact	Contact Updated Successfully
DELETE	/contact/:id	Delete contact	Contact Deleted Successfully

9. Create Contact – POST /contact

```
app.post("/contact", async (req, res) => {
   try {
      const newContact = new Contact({
         name: req.body.name,
         email: req.body.email,
         message: req.body.message
      });
      await newContact.save();
      res.send("Contact Saved Successfully");
   } catch (error) {
      console.log(error);
      res.status(500).send("Error Saving Contact");
   }
});
```

The React contact form sends name, email and message as JSON. The backend creates a Contact document and saves it in MongoDB.

10. Read Contacts – GET /contacts

```
app.get("/contacts", async (req, res) => {
  try {
    const contacts = await Contact.find();
    res.json(contacts);
  } catch (error) {
    console.log(error);
    res.status(500).send("Error Fetching Contacts");
  }
});
```

This endpoint retrieves all contact records and returns them as JSON. It is used by the admin panel to load customer messages.

11. Update Contact – PUT /contact/:id

```
app.put("/contact/:id", async (req, res) => {
  try {
    await Contact.findByIdAndUpdate(
      req.params.id,
      {
        name: req.body.name,
        email: req.body.email,
        message: req.body.message
      },
      { new: true }
    );
    res.send("Contact Updated Successfully");
  } catch (error) {
    console.log(error);
    res.status(500).send("Update Failed");
  }
});
```

The selected contact is identified by its MongoDB _id. The submitted name, email and message replace the existing values.

12. Delete Contact – DELETE /contact/:id

```
app.delete("/contact/:id", async (req, res) => {
  try {
    await Contact.findByIdAndDelete(req.params.id);
    res.send("Contact Deleted Successfully");
  } catch (error) {
    console.log(error);
    res.status(500).send("Delete Failed");
  }
});
```

The delete endpoint receives a contact ID in the URL and removes the matching MongoDB document.

13. Server Startup

```
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

The backend listens on port 5000 and displays the server address in the terminal.

14. Frontend–Backend Communication

1. The user fills in the Contact form.
2. React sends a POST request to http://localhost:5000/contact.
3. Express receives the JSON request.
4. Mongoose saves the Contact document in MongoDB.
5. The backend returns a success or error response.
6. The admin panel requests GET /contacts to display saved messages.
7. PUT /contact/:id updates a selected record.
8. DELETE /contact/:id removes a selected record.

15. Backend Data Flow

```
React Contact Form
      |
      | POST /contact
      v
Express.js API Server
      |
      | Mongoose
      v
MongoDB (nexatech)
      |
      | GET /contacts
      v
Admin Panel
      |
      +---- PUT /contact/:id
      |
      +---- DELETE /contact/:id
```

16. Error Handling

- MongoDB connection errors are printed in the backend terminal.
- Save failures return HTTP 500.
- Fetch failures return HTTP 500.
- Update failures return HTTP 500.
- Delete failures return HTTP 500.
- The frontend can display an error message when the server cannot be reached.

17. .env and .gitignore

The .env file is intended for environment-specific configuration. The .gitignore file should exclude sensitive and generated files before the project is uploaded to GitHub.

```
# Recommended .gitignore entries
node_modules/
.env
```

18. Running the Backend

9. Open the backend folder in the terminal.
10. Run npm install if dependencies are not installed.
11. Make sure MongoDB is running locally.
12. Start the server with the project's configured command, for example: node server.js.
13. Confirm the MongoDB connection message appears.
14. Confirm the server is running on port 5000.
15. Keep the backend server running while testing the frontend.

19. Testing Checklist

- MongoDB connection works successfully.
- Server starts on port 5000.
- POST /contact saves a contact.
- GET /contacts returns contacts.
- PUT /contact/:id updates a contact.
- DELETE /contact/:id deletes a contact.
- React frontend communicates with the API.
- Admin panel loads stored contacts.

20. Project Status

Feature	Status
Express server	Completed
CORS configuration	Completed
JSON request handling	Completed
MongoDB connection	Completed
Contact schema/model	Completed
Create contact API	Completed
Read contacts API	Completed
Update contact API	Completed
Delete contact API	Completed
Frontend API communication	Completed

21. Conclusion

The NexaTech Project 2 backend provides the server-side foundation for contact management. Node.js and Express.js handle API requests, while MongoDB and Mongoose provide persistent storage. The backend implements the CRUD operations required by the frontend and admin panel, making it a complete backend component for the NexaTech application.